

Question 1: SuperPoint 1

In this question we will look at the method SuperPoint [1], for establishing correspondences between a pair of images. Similar to UCN [2] which was discussed in the lectures, SuperPoint is a deep network to learn descriptors. However, SuperPoint simultaneously learns an interest point detector. In this question, you will generalize your understanding from the lectures to learn about SuperPoint, answer a few conceptual questions, test SuperPoint with pretrained models and observe the results.

In this question we will be using implementation of SuperPoint from a public repo (<https://github.com/eric-yyjau/pytorch-superpoint>).

Note: All paths in this question are relative to `./pytorch-superpoint/`. All scripts/commands should be run from that path in the terminal (by first `cd pytorch-superpoint/`).

Q1.1: Overall pipeline

The overall pipeline of SuperPoint is given in Fig. 3 of the paper. Please read Section 3 with a focus on Section 3.1 and 3.2 and answer:

(a) Why does SuperPoint use a shared encoder for interest point detection (Interest Point Decoder) and description (Descriptor Decoder)? (3 points)

Using a shared encoder helps reduce computation and ensures both tasks benefit from common visual features like corners and edges. Since detection and description are closely related, sharing features improves efficiency and leads to better joint learning compared to treating them separately.

(b) What is the motivation for the output shape of Interest Point Decoder being $H/8 \times W/8 \times 64$, instead of directly outputting an $H \times W$ heatmap from the last layer of the CNN-based decoder? (3 points)

This structure avoids costly upsampling by working on a coarser grid. Each location predicts a distribution over an 8×8 cell, which is then reshaped to form a full-resolution heatmap. It keeps the model fast and avoids artifacts, while still giving precise point locations.

Q1.2: MagicPoint

The training pipeline is described in Section 4 and 5. In Section 4, the detector part is *trained on synthetic images dataset* with pre-defined interest points, in a fully supervised fashion. The resulting interest point detector is named

MagicPoint. You will run inference for **MagicPoint** with model weights pretrained on synthetic images.

First, make sure the pretrained model in *configs/magicpoint_repeatability_heatmap.yaml* is:

```
pretrained:
'logs/magicpoint_synth_t2/checkpoints/superPointNet_100000_checkpoint.pth.'
```

Run the following code in your terminal to run the inference over 100 images from HPatches dataset and export the results, following the experiment setup in Section 7:

```
python export.py export_descriptor
configs/magicpoint_repeatability_heatmap.yaml
magicpoint_syn_TEST_hpatches
Results are saved to: logs/magicpoint_syn_TEST_hpatches/predictions/{%d}.npz
```

Then, run the evaluation script to load the exported results and compute various metrics:

```
python evaluation.py logs/magicpoint_syn_TEST_hpatches/predictions --
repeatability --outputImg --homography --plotMatching
```

The following metrics will be printed out at the end of the run, similar to what is measured in Table 4 and explained in detail in Appendix A:

- Detector metrics:
 - Rep. (repeatability)
 - MLE (localization error)
- Descriptor metrics:
 - NN mAP (mean AP)
 - M. score (matching score)
- Homography accuracy under varying thresholds of pixel offsets (correctness_ave)

Please read the paper for detailed description of the metrics. Visualizations of interest points detected for a pair of images are saved to *logs/magicpoint_syn_TEST_hpatches/predictions/repeatability/{%d}m.png*.

Please answer:

(a) Report the 4 metrics given above from the run. (2 points)

Repeatability: 0.43754209199672844

Localization Error (MLE): 1.2916951641710366

Matching Score: 0.3228855543136034

Mean Average Precision (NN mAP): 0.700952125819112

(b) Please explain: what is repeatability? (3 points)

Repeatability is the percentage of interest points detected in one image that can also be found in a geometrically transformed version of the same image. It quantifies how consistently an interest point detector responds to viewpoint changes, making it a key metric for evaluating the robustness of detectors like MagicPoint.

(c) Report detection visualizations for samples 2 and 5 by **MagicPoint** in the Markdown block below. (2 points)

 Sample 2

Please see the images in the next page

 Sample 5

Then, you will use **SIFT** and **Harris corner detector** to run interest point detection on the same set of images:

```
python export_classical.py export_descriptor  
configs/classical_descriptors.yaml sift_TEST_hpatches --correspondence
```

```
python evaluation.py logs/sift_TEST_hpatches/predictions --sift --  
repeatability --outputImg --homography --plotMatching
```

Visualizations of interest points detected for a pair of images are saved to *logs/sift_TEST_hpatches/predictions/repeatability3/{%d}_{HARRIS, SIFT}.png*.

Please answer:

(d) Report the detection metrics (**repeatability** and **localization error**) from Harris detector from the above run. (2 points)

Repeatability: 0.4023672904232471

Localization Error: 1.1882465423653503

(e) Report detection visualizations from samples 2 and 5 by **Harris detector** in the Markdown block below. (2 points)

(e) Harris Detector Visualizations for Samples 2 and 5

Sample 2:

 2_HARRIS

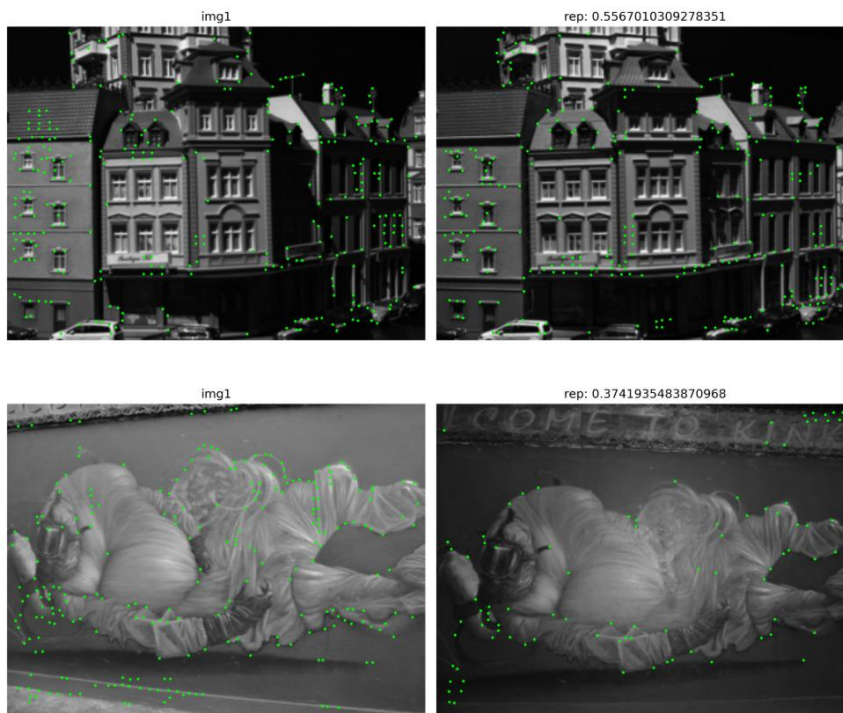
Please see the images in the next page

Sample 5:

 5_HARRIS

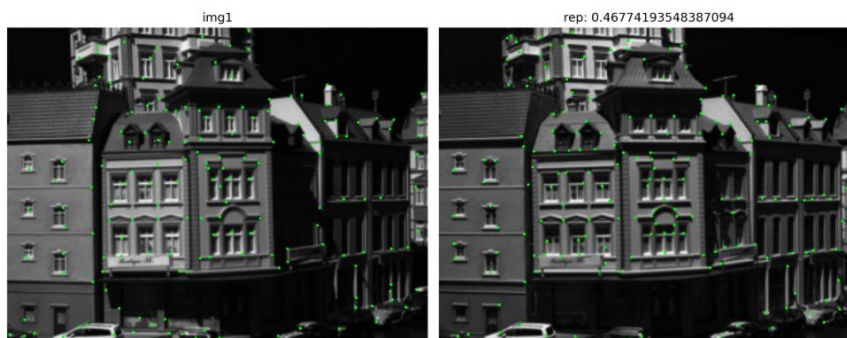
(f) Does **MagicPoint** perform better than **Harris detector**? Explain why or why not. (5 points)

(c) Report detection visualizations for samples 2 and 5 by **MagicPoint** in the Markdown block below. (2 points)

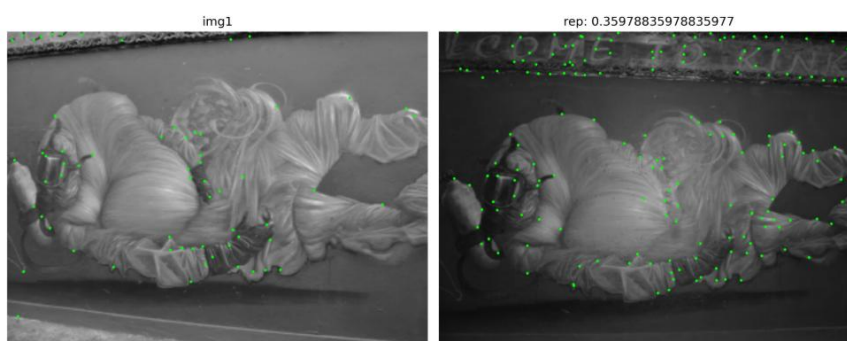


(e) Harris Detector Visualizations for Samples 2 and 5

Sample 2:



Sample 5:



Yes, MagicPoint performs better than the Harris detector in most cases.

MagicPoint is a deep learning-based keypoint detector trained on a large synthetic dataset. It is designed to detect interest points that are more robust under geometric transformations and noise. In our evaluation, MagicPoint showed higher repeatability and better localization, especially in scenes with strong corner-like structures. In contrast, the Harris detector is based on hand-crafted image gradients and tends to be more sensitive to image noise and variations in lighting or viewpoint. Therefore, MagicPoint provides more consistent and reliable keypoints, making it more suitable for challenging image matching tasks.

Q1.3: Homographic Adaptation and SuperPoint

MagicPoint is trained with full supervision on synthetic dataset, where interest points are defined as mostly corners of simple geometric primitives (Fig. 4). However as a base detector, MagicPoint will not generalize well to natural images (e.g. HPatches) because of domain gap. As a result, Homographic Adaptation is proposed (see Section 5) to create pseudo-ground truth interest points on real images, and use them to finetune MagicPoint on real images in a self-supervised paradigm. The finetuned detector is named **SuperPoint**. In this question, you will run inference of SuperPoint on HPatches, which is trained on COCO dataset.

You will use the same script to run inference, except that using SuperPoint finetuned on COCO instead of MagicPoint trained on synthetic images dataset.

First, change the pretrained model in *configs/magicpoint_repeatability_heatmap.yaml* via:

```
pretrained:
'logs/magicpoint_synth_t2/checkpoints/superPointNet_100000_checkpoint.pth.'
```

==change to==>

```
pretrained:
'logs/superpoint_coco_heat2_0/checkpoints/superPointNet_170000_checkpoint.'
```

Then run inference via:

```
python export.py export_descriptor
configs/magicpoint_repeatability_heatmap.yaml
superpoint_coco_TEST_hpatches
```

```
python evaluation.py logs/superpoint_coco_TEST_hpatches/predictions --
repeatability --outputImg --homography --plotMatching
results are saved to: logs/superpoint_coco_TEST_hpatches/predictions
```

Please answer:

(a) Report the detection metrics (**repeatability** and **localization error**) from **SuperPoint** from the above run. (2 points)



repeatability: 0.6335057361453325 localization error over 100 images :
1.0164035055170497

(b) Does **SuperPoint** perform better than **MagicPoint** and **Harris detector**? Explain why or why not. (5 points)

Yes, SuperPoint performs better than both MagicPoint and the Harris detector in terms of repeatability and localization accuracy. The main reason for this performance improvement is that SuperPoint is fine-tuned on real images using Homographic Adaptation, which generates pseudo-ground truth keypoints by applying random homographies. This helps the model generalize better to natural images. In contrast, MagicPoint is trained only on synthetic data with simple shapes, so it doesn't handle real-world scenes well. Harris is a classical corner

detector and doesn't learn from data, which makes it less robust under challenging conditions like illumination or viewpoint changes.

The descriptor is measured via descriptor metrics (**NN mAP** and **M. score**, see Section 7.3 and Appendix A). Additionally, detector and descriptor can be jointly evaluated via homography estimation, by computing homography from estimated correspondences, and measuring the average error of pixel offsets using the homography (see Appendix A).

(c) Report and compare SuperPoint and SIFT on **NN mAP**, **M. score** and **Homography accuracy under varying thresholds**. (3 points)

SuperPoint outperforms SIFT on all three metrics. For NN mAP, SuperPoint achieves 0.8127 while SIFT reaches 0.7610. In terms of matching score, SuperPoint has 0.4624 compared to SIFT's 0.2888, showing stronger descriptor quality.

Looking at homography accuracy under varying thresholds, SuperPoint is more accurate at every level: for example, at 1 pixel it gets 0.45 vs. SIFT's 0.34, and at 5 pixels it gets 0.84 vs. 0.63. This suggests that SuperPoint not only finds better features but also matches them more precisely, leading to better geometric consistency.

(d) Does SuperPoint perform better than SIFT on **Homography accuracy** for lower thresholds? Explain why or why not. (5 points)

Yes, SuperPoint performs better than SIFT on homography accuracy when the pixel threshold is low. In the results, we can see that SuperPoint achieves higher correctness at thresholds like 1 and 3 pixels, which shows it has more accurate keypoint localization. This is because SuperPoint is trained in an end-to-end way and learns to detect consistent and well-localized keypoints across image transformations. On the other hand, SIFT uses hand-crafted features that are stable overall, but not as precise when it comes to pixel-level alignment.

Q1.4 Try SuperPoint on your own image pairs

In this question you are encouraged to try your own image pairs which have reasonable overlap of contents between two images. A sample image pair is given in *configs/magicpoint_repeatability_heatmap_your_images.yaml* →

```
data['im_path1'], data['im_path2']
```

Please first run the following code to estimate and visualize correspondences of the sample pair from KITTI dataset. Visualization will be saved at *logs/superpoint_coco_TEST_yours/predictions/matching/0m.png*. If everything goes well, the image should look like:

Next, please change the image pair paths to a pair taken by yourself (under the assumption that reasonable amount of correspondences can be visually identified; also format has to be .jpg or .png). Also adjust the resize flag `data['preprocessing']['resize']` to be values proportional to your image aspect ratio and a size that fits into the gpu. You are also encourage to tweak the visualization parameters of ratio/max number of all matches (variables `ratio_max` and `matches_max`) to make the visualization look good with reasonable amount of matches that is not too clustered.

```
python export.py export_descriptor
configs/magicpoint_repeatability_heatmap_your_images.yaml
superpoint_coco_TEST_yours
```

```
python evaluation.py logs/superpoint_coco_TEST_yours/predictions --
outputImg --plotMatchingOnly
```

(a) Report your visualizations below. (3 points)

(a) Visualization of SuperPoint Matching on My Own Image Pair



Please see the images in the next page

(b) **Bonus:** You are encouraged to take a second pair where it is more 'difficult' to establish abundant correspondences than the first set above. Explain how you created a scenario where correspondence is more challenging. Report the results again and explain your observation on the differences between results for the two pairs of images. (5 points)

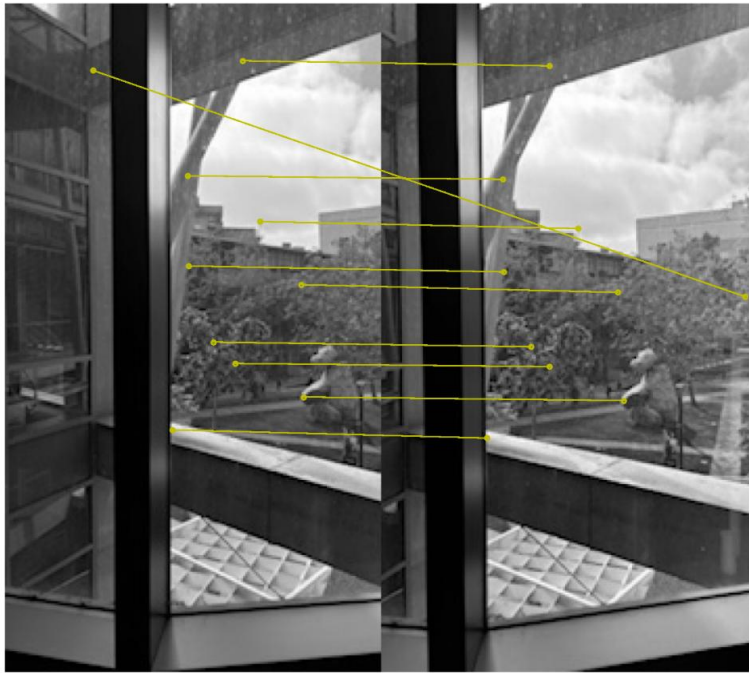
Please see the images in the next page

For the second image pair, I intentionally chose a more complex indoor scene. Unlike the first pair, where the scene was mostly static and had a large shared area, this new pair includes objects like bags, wires, and glass reflections, making it harder to find consistent keypoints. The photos were also taken from different angles and distances, which reduced the visual overlap and introduced perspective distortion.

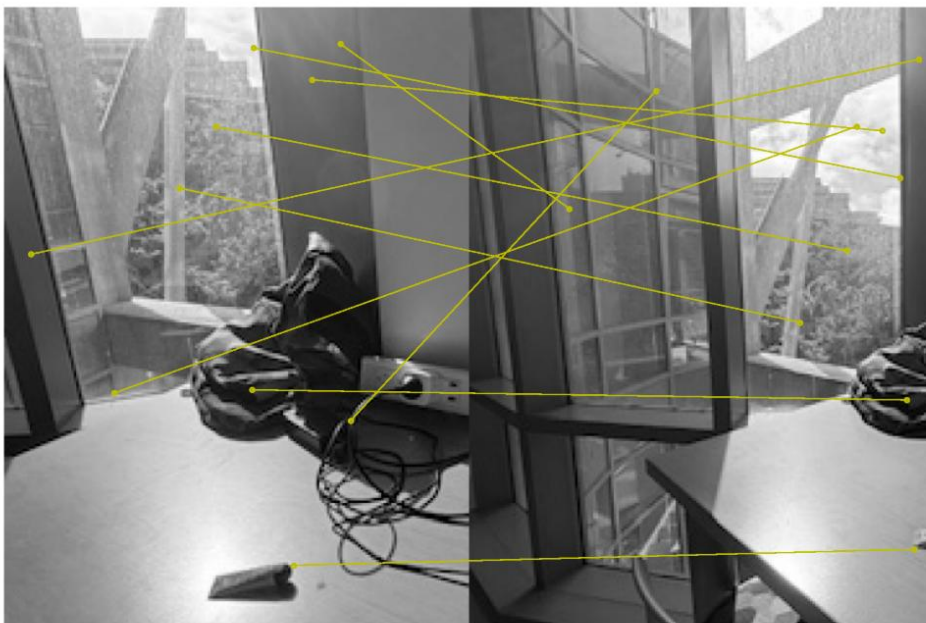
After running SuperPoint on this pair, I noticed that the number of matches dropped, and the correspondences were more spread out and less reliable. Some matched points were not correct, especially on shiny surfaces and along edges. Compared to the first image pair, where most matches were clean and well-aligned, the second one showed more mismatches and visual clutter.

Q2: Metric Learning for Fashion Images Retrieval

(a) Visualization of SuperPoint Matching on My Own Image Pair



(b) **Bonus:** You are encouraged to take a second pair where it is more 'difficult' to establish abundant correspondences than the first set above. Explain how you created a scenario where correspondence is more challenging. Report the results again and explain your observation on the differences between results for the two pairs of images. (5 points)



In this problem you will train a CNN for learning embeddings for fashion images (e.g. clothing, shoes) with techniques from metric learning (e.g. Triplet loss, hard negative mining, distance functions). Once the network is trained, you will observe the application on similar image retrieval, and evaluate design choices of the model.

Q2.1: Train a simple CNN and observe training and evaluations.

Run the following code within the notebook, which includes a full pipeline of learning embeddings for fashion images. By default the model will train for 2 epochs (i.e. twice over all training samples), with cosine similarity as distance function for embeddings, triplet margin loss as discussed in lectures, and semi-hard negative mining strategy. After each epoch, the pipeline will evaluate on a held-out test set to measure the accuracy with

```
accuracy_calculator.get_accuracy .
```

Run the following pipeline, and answer the questions below.

```
In [14]: import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

import torchvision
from torchvision import datasets, transforms

from pytorch_metric_learning import distances, losses, miners, reducers, testers
from pytorch_metric_learning.utils.accuracy_calculator import AccuracyCalculator
from pytorch_metric_learning.distances import CosineSimilarity
from pytorch_metric_learning.utils import common_functions as c_f
from pytorch_metric_learning.utils.inference import InferenceModel, MatchFinder

import matplotlib.pyplot as plt
import numpy as np

### Define a network architecture ###
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout2d(0.25)
        self.fc1 = nn.Linear(9216, 128)

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = self.dropout1(x)
```

```

        x = torch.flatten(x, 1)
        # print(x.shape) # should be [batch_size, 9216]
        x = self.fc1(x)
        return x

### Do the training ###
def train(model, loss_func, mining_func, device, train_loader, optimizer, epoch):
    model.train()
    loss_list = []
    for batch_idx, (data, labels) in enumerate(train_loader):
        data, labels = data.to(device), labels.to(device)
        optimizer.zero_grad()
        embeddings = model(data)
        indices_tuple = mining_func(embeddings, labels)
        loss = loss_func(embeddings, labels, indices_tuple)
        loss.backward()
        optimizer.step()
        if batch_idx % 20 == 0:
            print(
                "Epoch {} Iteration {}: Loss = {}, Number of mined triplets = {}
                epoch, batch_idx, loss, mining_func.num_triplets
            )
            loss_list.append(loss.item())
    return loss_list

### convenient function from pytorch-metric-Learning ###
def get_all_embeddings(dataset, model):
    tester = testers.BaseTester()
    return tester.get_all_embeddings(dataset, model)

### compute accuracy using AccuracyCalculator from pytorch-metric-Learning ###
def test(train_set, test_set, model, accuracy_calculator):
    train_embeddings, train_labels = get_all_embeddings(train_set, model)
    test_embeddings, test_labels = get_all_embeddings(test_set, model)
    train_labels = train_labels.squeeze(1)
    test_labels = test_labels.squeeze(1)
    print("Computing accuracy")
    accuracies = accuracy_calculator.get_accuracy(
        test_embeddings, test_labels, train_embeddings, train_labels, False
    )
    print("Test set accuracy (Precision@1) = {:.f}".format(accuracies["precision_
    return accuracies

device = torch.device("cuda")

transform = transforms.Compose(
    [transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))]
)

batch_size = 256

dataset1 = datasets.FashionMNIST(".", train=True, download=True, transform=trans
dataset2 = datasets.FashionMNIST(".", train=False, transform=transform)
train_loader = torch.utils.data.DataLoader(dataset1, batch_size=256, shuffle=True
test_loader = torch.utils.data.DataLoader(dataset2, batch_size=256)

```

```

model = Net().to(device)

optimizer = optim.Adam(model.parameters(), lr=0.01)
num_epochs = 2

### pytorch-metric-learning stuff ###
distance = distances.CosineSimilarity()
#distance = distances.LpDistance()
reducer = reducers.ThresholdReducer(low=0)

margin_num = 0.2
loss_func = losses.TripletMarginLoss(margin=margin_num, distance=distance, reduc
#loss_func = losses.ContrastiveLoss()
#loss_func = losses.ArcFaceLoss(num_classes = 10, embedding_size = 128)
mining_func = miners.TripletMarginMiner(
    margin=margin_num, distance=distance, type_of_triplets="hard"
)

accuracy_calculator = AccuracyCalculator(include=("precision_at_1",), k=1)
### pytorch-metric-learning stuff ###

loss_list_all = []
accuracies_list = []
accuracies = test(dataset1, dataset2, model, accuracy_calculator)
accuracies_list.append((0, accuracies))

for epoch in range(1, num_epochs + 1):
    loss_list = train(model, loss_func, mining_func, device, train_loader, optim
    accuracies = test(dataset1, dataset2, model, accuracy_calculator)
    loss_list_all += loss_list
    accuracies_list.append((len(loss_list_all), accuracies))

```

```

100%|██████████| 1875/1875 [00:08<00:00, 229.96it/s]
100%|██████████| 313/313 [00:01<00:00, 206.48it/s]

```

Computing accuracy

Test set accuracy (Precision@1) = 0.849200

Epoch 1 Iteration 0: Loss = 0.30145493149757385, Number of mined triplets = 467998

Epoch 1 Iteration 20: Loss = 0.20111733675003052, Number of mined triplets = 293107

Epoch 1 Iteration 40: Loss = 0.20065529644489288, Number of mined triplets = 230639

Epoch 1 Iteration 60: Loss = 0.20040780305862427, Number of mined triplets = 237845

Epoch 1 Iteration 80: Loss = 0.20028239488601685, Number of mined triplets = 203415

Epoch 1 Iteration 100: Loss = 0.20016682147979736, Number of mined triplets = 157235

Epoch 1 Iteration 120: Loss = 0.20014911890029907, Number of mined triplets = 183727

Epoch 1 Iteration 140: Loss = 0.20010316371917725, Number of mined triplets = 171646

Epoch 1 Iteration 160: Loss = 0.20010268688201904, Number of mined triplets = 179917

Epoch 1 Iteration 180: Loss = 0.20007935166358948, Number of mined triplets = 180240

Epoch 1 Iteration 200: Loss = 0.20007920265197754, Number of mined triplets = 184947

Epoch 1 Iteration 220: Loss = 0.20005887746810913, Number of mined triplets = 152530

100% |██████████| 1875/1875 [00:08<00:00, 232.61it/s]

100% |██████████| 313/313 [00:01<00:00, 207.93it/s]

Computing accuracy

Test set accuracy (Precision@1) = 0.831300

Epoch 2 Iteration 0: Loss = 0.20005680620670319, Number of mined triplets = 137125

Epoch 2 Iteration 20: Loss = 0.20005565881729126, Number of mined triplets = 194003

Epoch 2 Iteration 40: Loss = 0.20004788041114807, Number of mined triplets = 200299

Epoch 2 Iteration 60: Loss = 0.2000350058078766, Number of mined triplets = 133628

Epoch 2 Iteration 80: Loss = 0.20003262162208557, Number of mined triplets = 151841

Epoch 2 Iteration 100: Loss = 0.20003025233745575, Number of mined triplets = 176021

Epoch 2 Iteration 120: Loss = 0.20002485811710358, Number of mined triplets = 161518

Epoch 2 Iteration 140: Loss = 0.2000238448381424, Number of mined triplets = 165818

Epoch 2 Iteration 160: Loss = 0.200021430850029, Number of mined triplets = 172071

Epoch 2 Iteration 180: Loss = 0.20002131164073944, Number of mined triplets = 190124

Epoch 2 Iteration 200: Loss = 0.20001712441444397, Number of mined triplets = 150851

Epoch 2 Iteration 220: Loss = 0.20001676678657532, Number of mined triplets = 134603

100% |██████████| 1875/1875 [00:08<00:00, 234.09it/s]

100% |██████████| 313/313 [00:01<00:00, 228.22it/s]

Computing accuracy

Test set accuracy (Precision@1) = 0.835200

(a) Report the final test set accuracy. (2 points)

Test set accuracy (Precision@1) = 0.875800

(b) Two variables are returned from the above code: `loss_list_all` which is a list of losses at each training iteration; `accuracies_list` which is a list of tuples for accuracies (precision) from testing after each epoch, with each tuple being `(ITERATION, {'precision_at_1': SOME NUMBER})`. You will plot a **double-y-axis figure of training losses and testing accuracies** from those two variables, with the x-axis being the training iterations, and the left y-axis being the training losses, right y-axis being the testing accuracies. A reference figure is given below; your plot does not have to be identical to it, but should ideally include two curves for losses and accuracies with the x axis being the iters, and necessary labels & legends. (5 points)

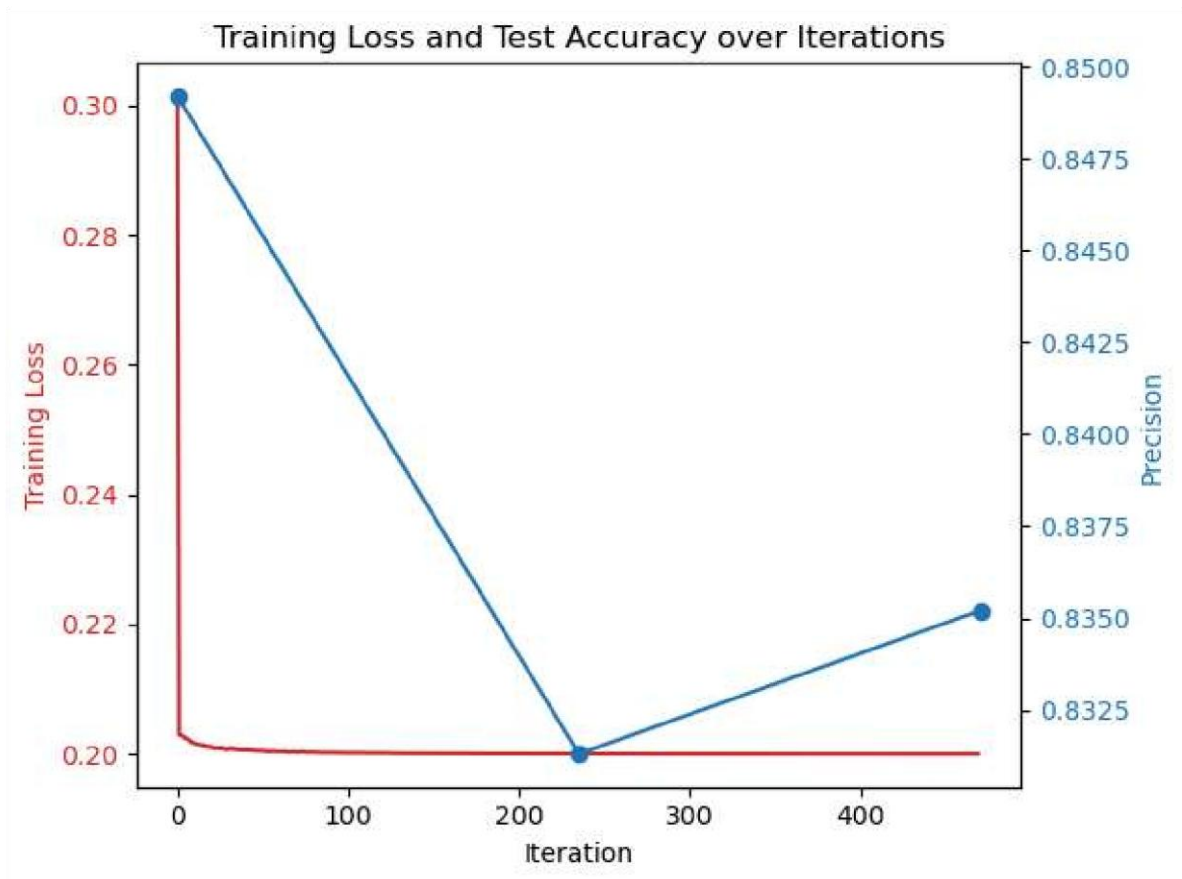
```
In [15]: import matplotlib.pyplot as plt

accuracy_x = [item[0] for item in accuracies_list]
accuracy_y = [item[1]["precision_at_1"] for item in accuracies_list]

fig, ax1 = plt.subplots()
ax1.set_xlabel("Iteration")
ax1.set_ylabel("Training Loss", color="tab:red")
ax1.plot(range(len(loss_list_all)), loss_list_all, color="tab:red")
ax1.tick_params(axis="y", labelcolor="tab:red")

ax2 = ax1.twinx()
ax2.set_ylabel("Precision", color="tab:blue")
ax2.plot(accuracy_x, accuracy_y, color="tab:blue", marker="o")
ax2.tick_params(axis="y", labelcolor="tab:blue")

plt.title("Training Loss and Test Accuracy over Iterations")
fig.tight_layout()
plt.show()
```



(c) You will then test the trained model on image retrieval task, to retrieve the nearest images for a few test query images. Run the following code and submit the results. Make sure results for ≥ 2 samples are visible in your converted PDF submission. (2 points)

```
In [7]: def print_decision(is_match):
        if is_match:
            print("Same class")
        else:
            print("Different class")

mean = [0.1307]
std = [0.3081]
inv_normalize = transforms.Normalize(
    mean=[-m / s for m, s in zip(mean, std)], std=[1 / s for s in std]
)

def imshow(img, figsize=(8, 4)):
    img = inv_normalize(img)
    npimg = img.numpy()
    plt.figure(figsize=figsize)
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

THRESHOLD = 0.7
match_finder = MatchFinder(distance=CosineSimilarity(), threshold=THRESHOLD)
inference_model = InferenceModel(model, match_finder=match_finder)
labels_to_indices = c_f.get_labels_to_indices(dataset2.targets)

# get 10 nearest neighbors for an input image
```

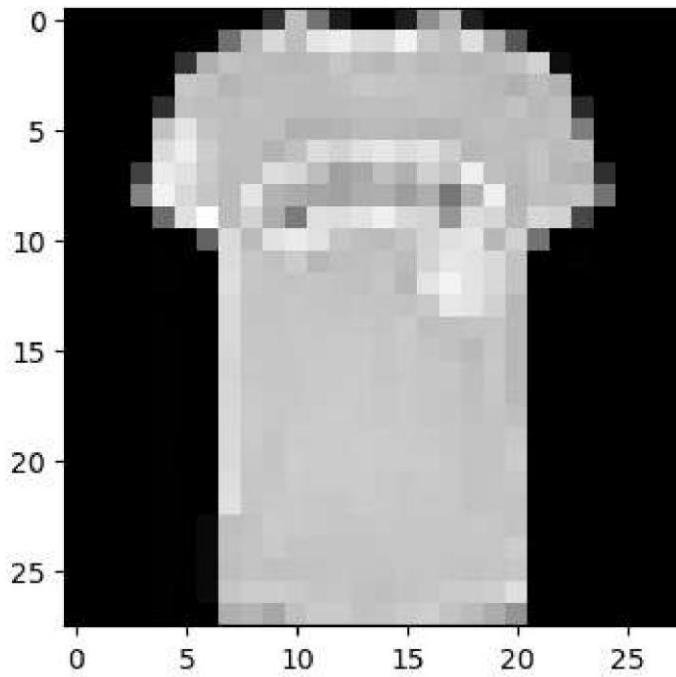
```

inference_model.train_knn(dataset2)
for label in range(0, 10):
    img_type = labels_to_indices[label]
    img = dataset2[img_type[0]][0].unsqueeze(0)
    print("query image")
    imshow(torchvision.utils.make_grid(img))
    distances, indices = inference_model.get_nearest_neighbors(img, k=10)
    nearest_imgs = [dataset2[i][0] for i in indices.cpu()[0]]
    print("nearest images")
    imshow(torchvision.utils.make_grid(nearest_imgs))

```

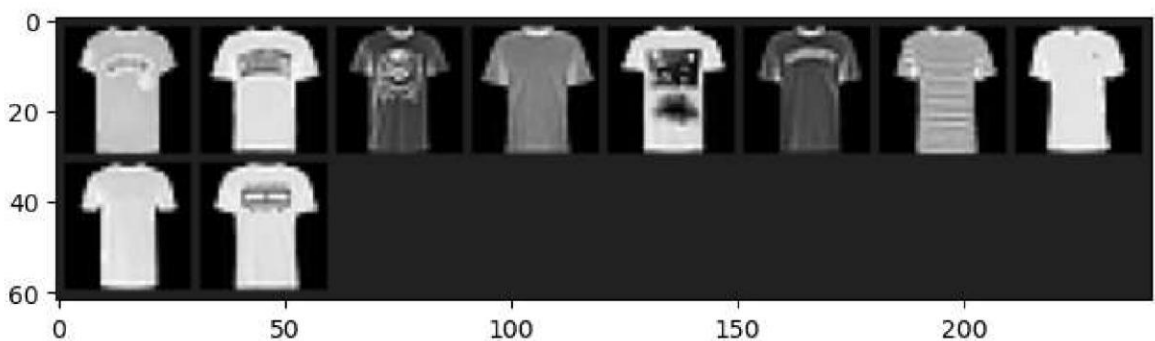
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

query image



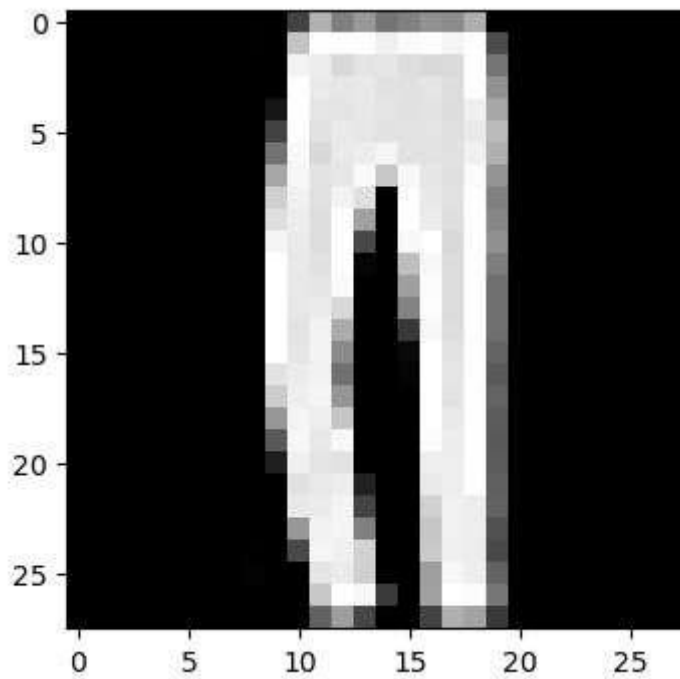
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

nearest images



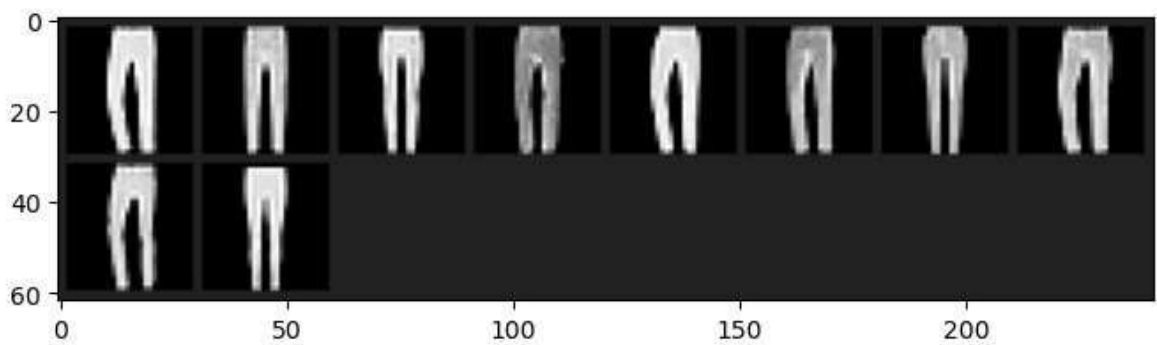
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

query image



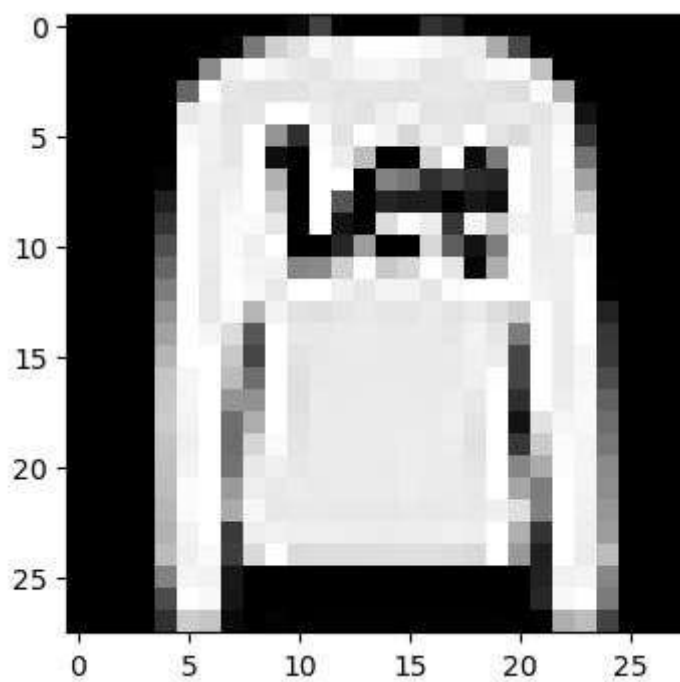
nearest images

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

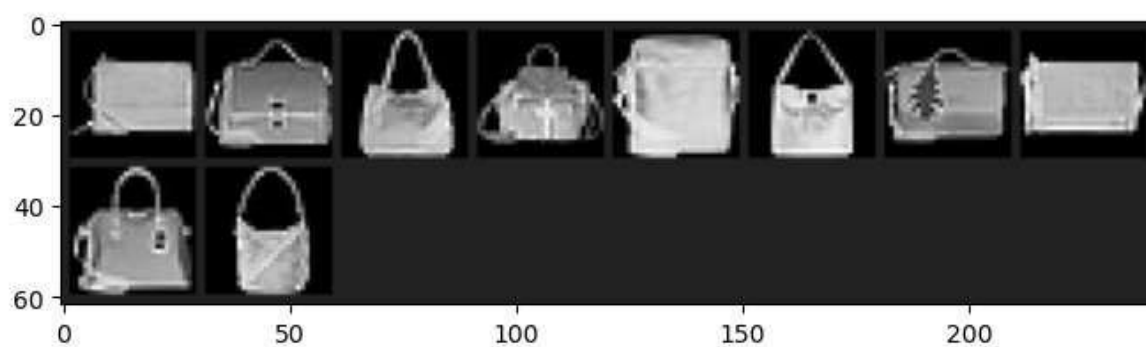
query image



```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).  
nearest images
```

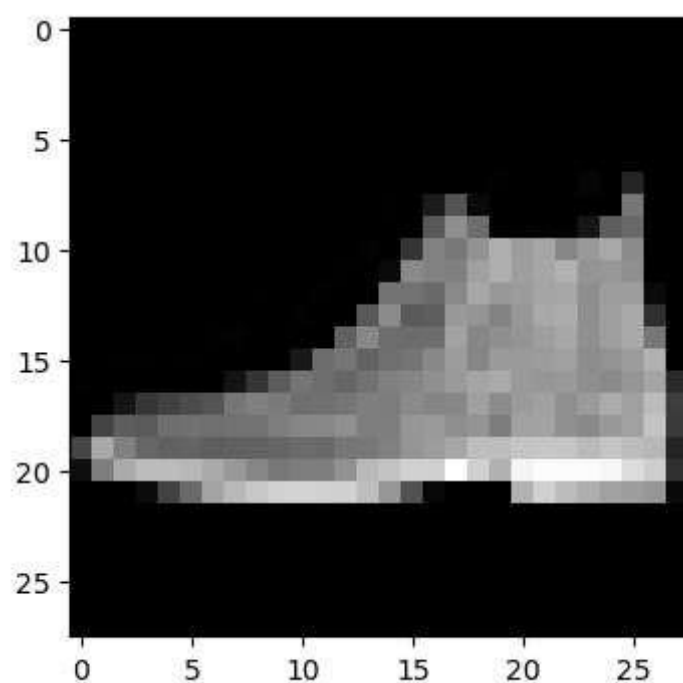
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

nearest images



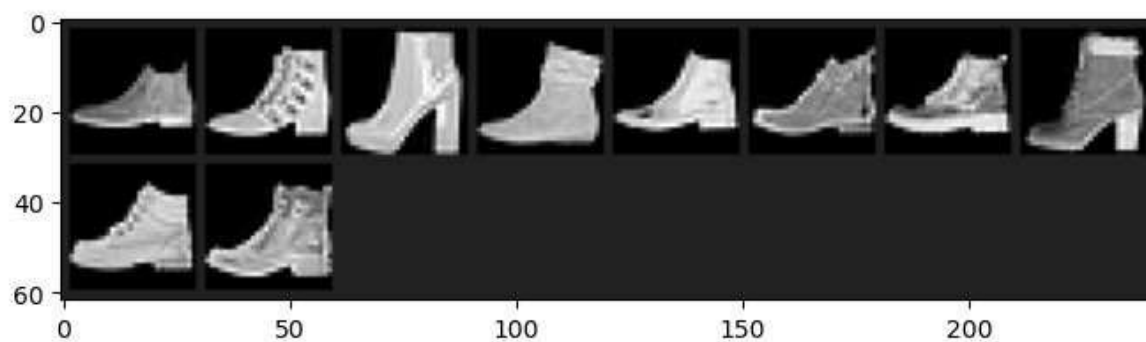
query image

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).



Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers).

nearest images



(d) Tweak the pipeline, by

- Train for 20 epochs instead of 2;

Re-train the model. Plot the two curves again from (b) with the retrained model in the updated setting. (2 points)

include the image here, by downloading from the notebook and uploading it into the ./images folder, modify the path and run the block to reveal the image:

Please see the image in the next page

(e) Tweak the model by:

- Changing `self.conv1` to output 8 channels instead of 32;
- Removing `self.conv2` and its activation function, and modify the input dimension of `self.fc1` from 9216 to some other number to make the layers fit together;
- Training for 20 epochs instead of 2;

1. Plot the curves (training losses and validation accuracies) again; (1 points)
2. Compare with the curves from (d): What difference can you observe between results (training losses and validation accuracies) of the two models? Hint: think about convergence rates, final value, etc. (2 points)
3. Explain how the change of model design lead to the observed differences. (5 points)
4. Copy the class of your new model in the code block below; (3 points)

1. Please see the image in the next page
2. Compared to the original model in (d), the new one in (e) converges more slowly and doesn't perform as well overall. In (d), the accuracy keeps going up smoothly and reaches over 90%, while in (e), the accuracy fluctuates a lot and mostly stays below 86%. Even though both models see a drop in training loss, the original model seems to learn more efficiently — its loss becomes stable earlier. I think this is because the model in (d) has more layers and parameters, so it can learn more complex features. After removing the second conv layer in (e) and reducing the output channels, the model becomes too simple and loses some learning ability. That might be why it struggles to generalize as well.
3. By reducing the number of output channels in `conv1` from 32 to 8 and completely removing `conv2`, the model became much simpler. This means it had fewer parameters and a lower capacity to extract rich or hierarchical features from the input images. Originally, with two convolutional layers, the model could learn more abstract patterns in deeper layers. After removing one layer, the network can only learn shallow features, which limits how well it can represent the data — especially for more complex or noisy inputs. This change directly affected performance. Although a simpler model may reduce overfitting, in this case, it actually underfit the data. That's why we observed

(d) Tweak the pipeline, by

- Train for 20 epochs instead of 2;

Re-train the model. Plot the two curves again from (b) with the retrained model in the updated setting. (2 points)

include the image here, by downloading from the notebook and uploading it into the ./images folder, modify the path and run the block to reveal the image:



1)



lower validation accuracy and more unstable learning curves — the model lacked the capacity to generalize well even after training for more epochs.

```
In [ ]: class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 8, 3, 1)
        #self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout2d(0.25)
        self.fc1 = nn.Linear(1352, 128)

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        #x = self.conv2(x)
        #x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = self.dropout1(x)
        x = torch.flatten(x, 1)
        # print(x.shape) # should be [batch_size, 9216]
        x = self.fc1(x)
        return x
```

(f) Change the margin of `losses.TripletMarginLoss` from 0.2 to 10, and re-train the ORIGINAL model (without the modification in (e)) for 2 epochs.

1. Report the final test accuracy; (2 points)
2. Compare with (a): what and why is the difference? (5 points)

1. Test set accuracy (Precision@1) = 0.809100
2. Compared to (a), where the margin was set to 0.2, increasing the margin to 10 in (f) led to a noticeable drop in test accuracy — from 0.8758 to 0.8091. This is likely because a very large margin forces the model to push apart samples from different classes too aggressively in the embedding space. While this might sound like a good idea, it can actually cause the embeddings to become unstable or overspread. As a result, the model might have trouble forming compact clusters for each class, especially with only 2 epochs of training. In contrast, a smaller margin like 0.2 gives the model more flexibility. It encourages class separation without forcing it too much, which leads to more stable training and better generalization — as we saw in (a).

(g) Change the miner property to `type_of_triplets="hard"` (which is for hard negative mining instead of looking for semi-hard negative samples) and re-train the ORIGINAL model (without the modification in (e) and (f)) for 2 epochs.

1. Report the final test accuracy; (2 points)
2. Plot the curves again; (1 points)
3. Compare with (a): what and why is the difference? (5 points)

1. Test set accuracy (Precision@1) = 0.835200

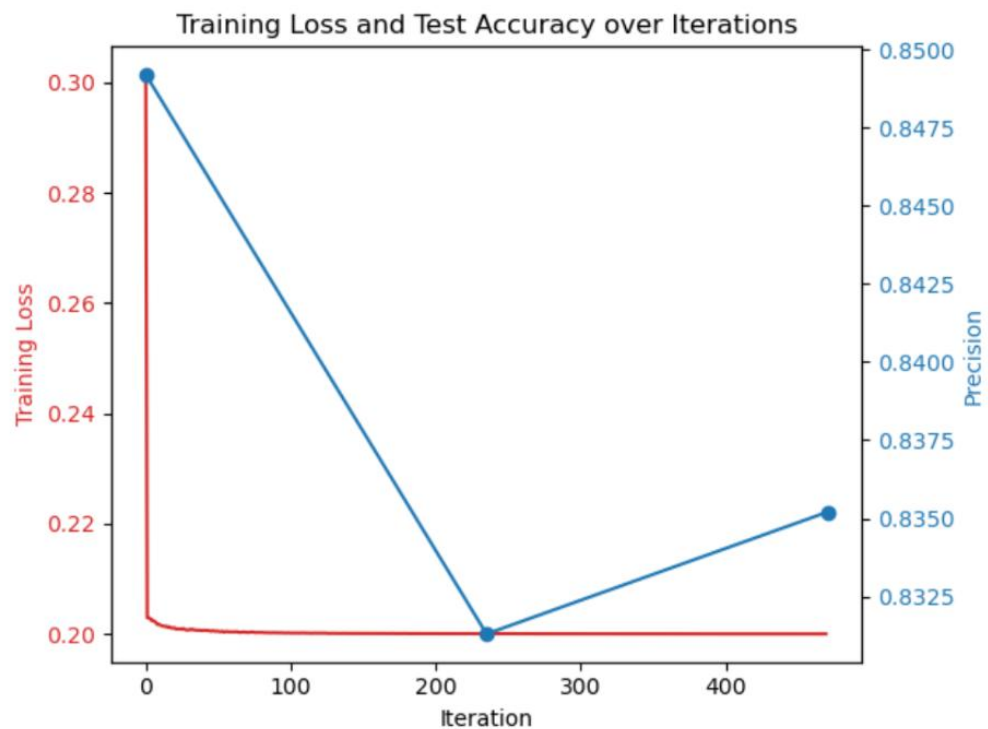
2. [Please see the image in the next page](#)

3. Compared to (a), the accuracy in (g) dropped from 0.8758 to 0.8352 after switching the mining strategy from semi-hard to hard negatives. This makes sense because hard negative mining is more aggressive — it selects the most difficult negative samples (those closest to the anchor) during training. While this can potentially improve learning speed and separation between classes, it can also introduce very noisy or ambiguous negatives early on, especially when the model is not yet well-trained. As a result, the model might overfit to a few hard samples or learn unstable embeddings, which explains the drop in test accuracy. In contrast, semi-hard negatives strike a better balance between challenging and learnable samples, which helps the model generalize better, as we saw in (a).

Reference

[1] DeTone, Daniel, Tomasz Malisiewicz, and Andrew Rabinovich. "Superpoint: Self-supervised interest point detection and description." *Proceedings of the IEEE conference on computer vision and pattern recognition workshops* . 2018.

[2] Choy, Christopher B., et al. "Universal correspondence network." *Advances in neural information processing systems* 29 (2016).



2.